

RUTA DETALLADA

PING-PONG_GAME

Unity 2D + Git Flow + assets + commits + cierre final

Objetivo: que puedas hacerlo solo, sin perderte, sabiendo qué crear, qué descargar, cuándo commitear, cuándo pushear y cuándo cerrar cada rama.

1. PREPARAR

Unity + repo

2. IMPORTAR

Assets + escena

3. PROGRAMAR

Gameplay + UI

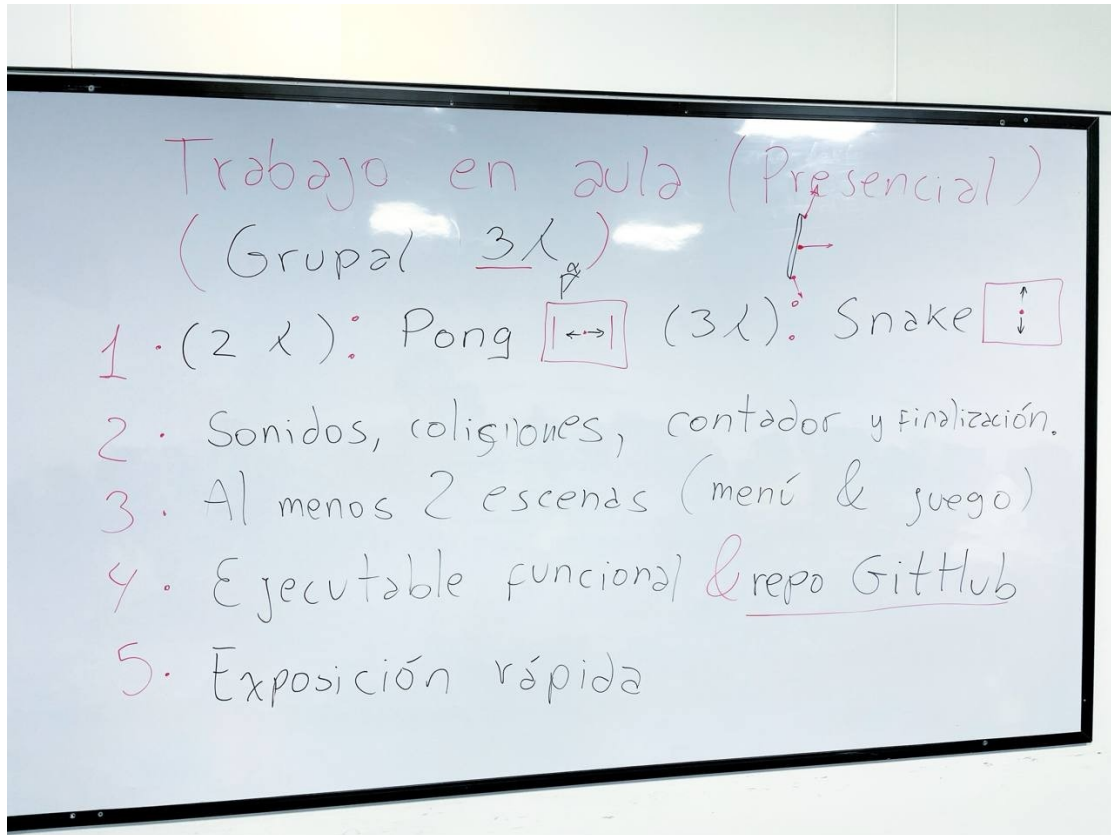
4. ENTREGAR

Build + release

Propuesta concreta del proyecto: haz un “Fantasy Pong”. La mecánica será la de Pong clásico, pero la presentación visual tendrá una estética pixel-art de bosque/fantasía para que se vea más trabajado, similar al ejemplo que te dieron.

Repositorio remoto objetivo: [Developer-vic1/Ping-Pong_Game](#)

1. Requisitos que debes cumplir (extraídos de la pizarra)



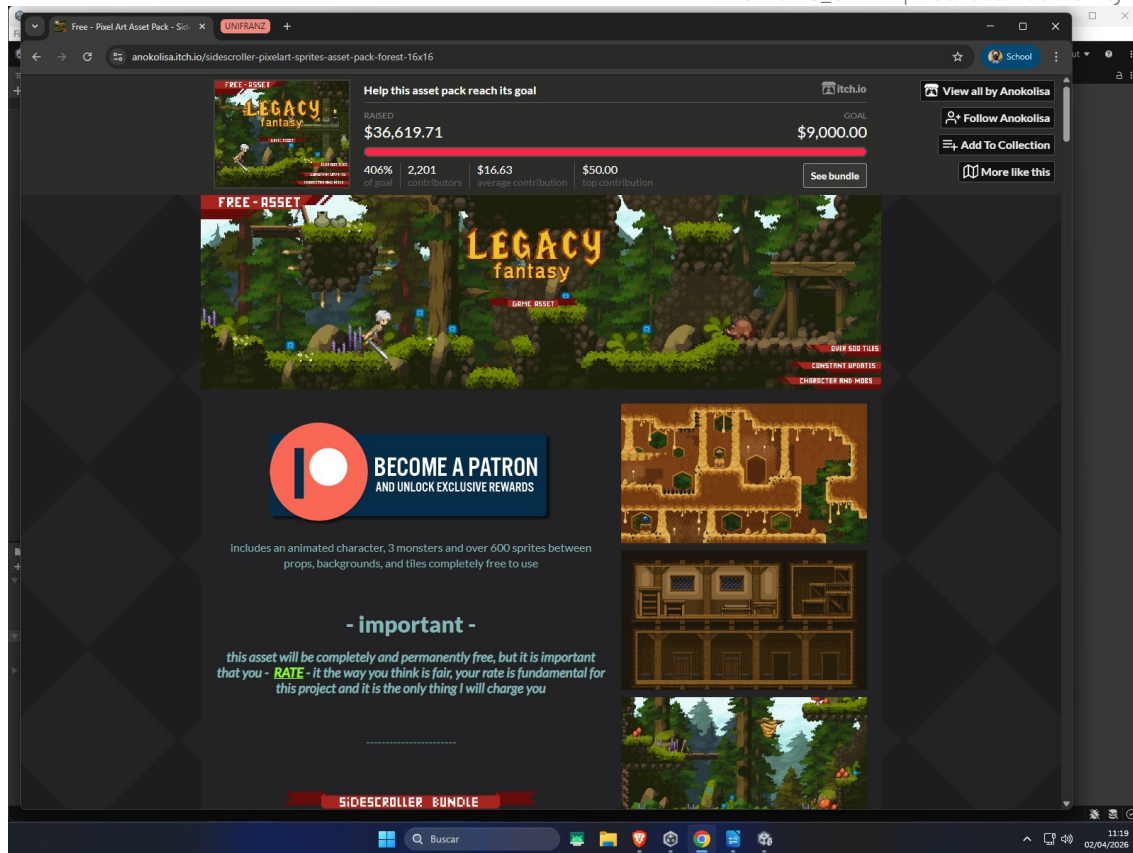
Fotografía original del trabajo en aula. A partir de esto se definió la ruta del proyecto.

1. Juego tipo Pong (o Snake). En tu caso elegimos Pong porque es más rápido de cerrar bien en Unity.
2. Debe tener sonidos, colisiones, contador/marcador y finalización de partida.
3. Debe tener al menos 2 escenas: Menú y Juego.
4. Debes entregar ejecutable funcional y repositorio en GitHub.
5. La exposición será rápida, por lo que el proyecto debe estar ordenado y fácil de explicar.

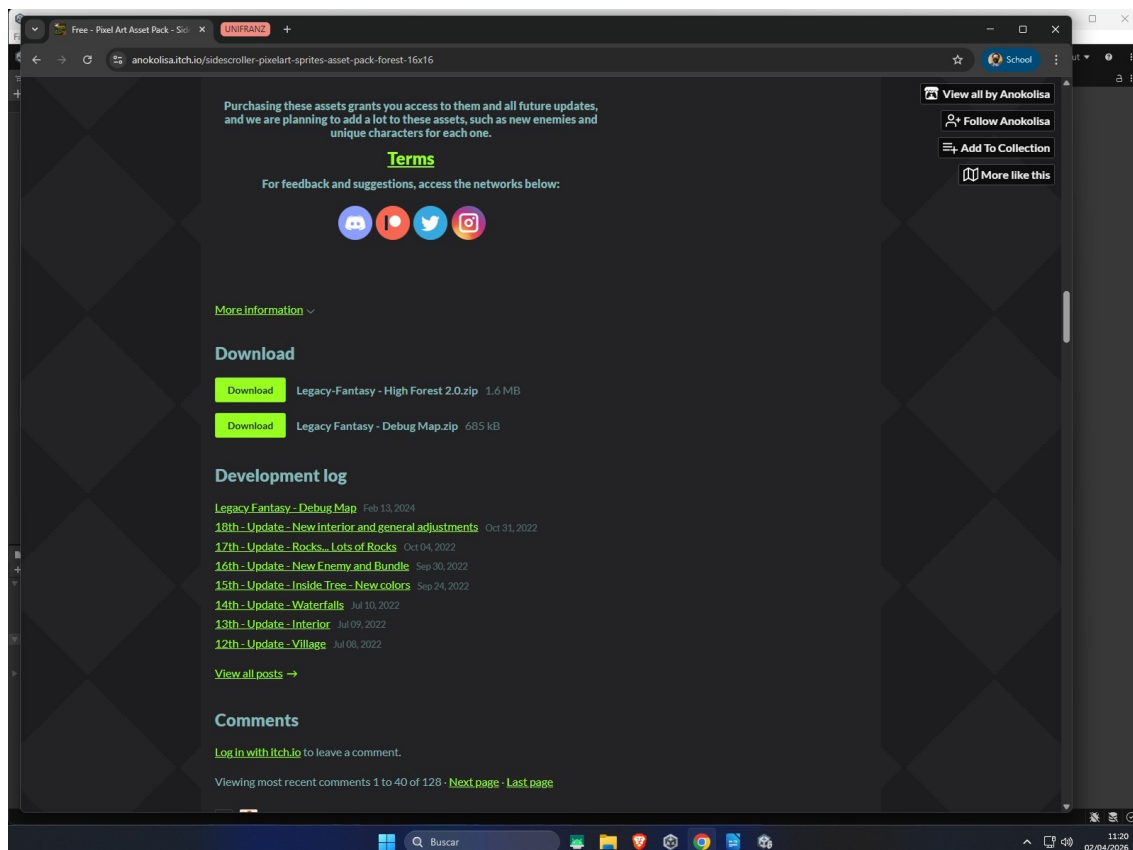
Meta realista: hacer un juego pequeño pero limpio y completo. Es mejor entregar un Pong bien cerrado que un proyecto más grande, roto o sin terminar.

2. Qué se hizo antes en el ejemplo y qué repetiremos

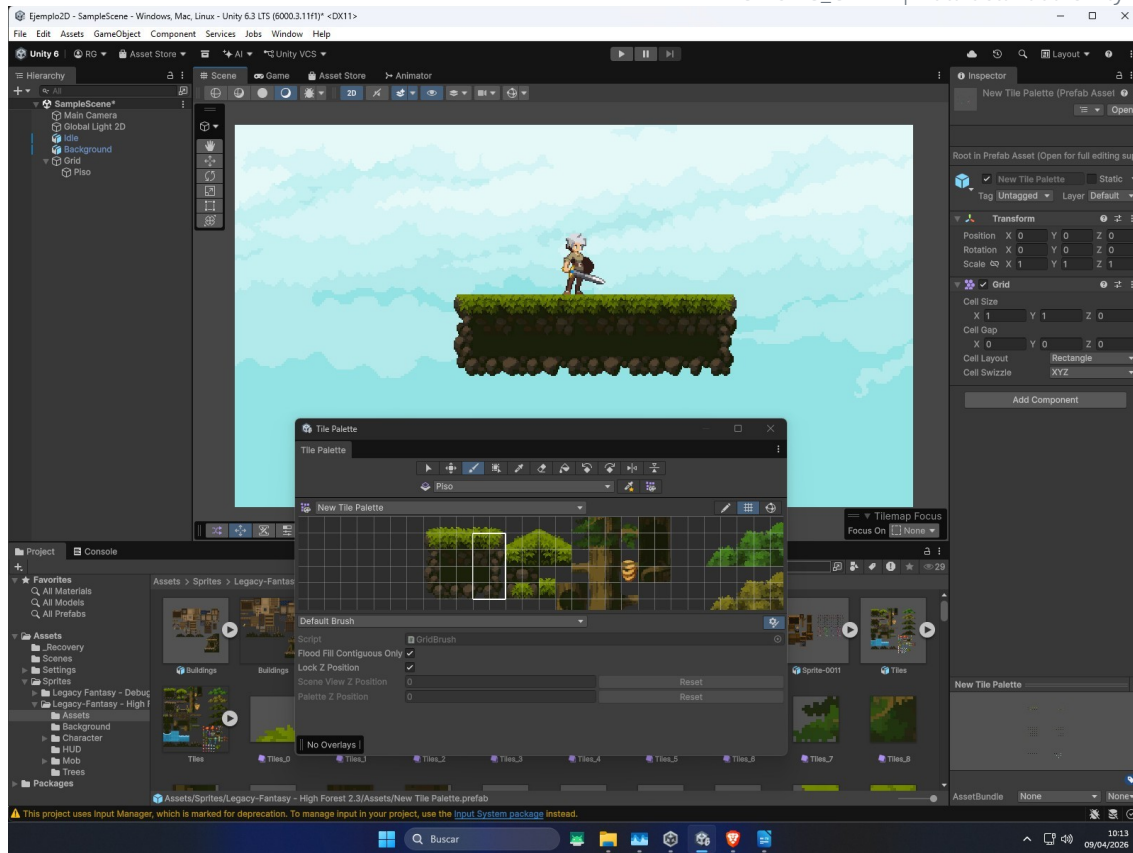
6. Buscar un paquete de arte 2D externo.
7. Descargar sprites y tiles.
8. Crear el proyecto y organizar carpetas.
9. Importar assets en Unity.
10. Construir primero el escenario / escena.
11. Asignar componentes (SpriteRenderer, Collider2D, Rigidbody2D).
12. Crear scripts C# y conectarlos a los objetos.
13. Probar en Play, corregir y recién después pulir.
14. Subir avances a GitHub por etapas.



Captura del ejemplo: página del asset pack Legacy Fantasy (estilo visual recomendado).



Captura del ejemplo: sección de descarga del asset pack usado como referencia.



Captura del ejemplo: construcción del escenario en Unity con Tile Palette / Tilemap.

Regla base de trabajo: no comiences escribiendo scripts al azar. Primero prepara el proyecto, luego importa assets, después arma la escena, luego programa, luego prueba, y recién al final cierras la release.

3. Assets recomendados y enlaces de descarga

Usaremos un set visual principal y recursos complementarios. No necesitas descargar 20 paquetes; con 3 o 4 buenos recursos ya es suficiente.

Tipo	Asset	Uso	Enlace
Visual principal	Legacy Fantasy - High Forest	Fondos, tiles, bordes, ambiente visual del bosque/fantasía.	Abrir asset
UI	Kenney Pixel UI Pack	Botones, paneles, elementos para menú / pausa / victoria.	Abrir asset
Audio UI	Kenney UI Audio	Clicks y sonidos de interfaz.	Abrir asset
Audio impactos	Kenney Impact Sounds	Golpes, rebotes, colisiones.	Abrir asset
Iconos de controles (opcional)	Kenney Input Prompts	Si quieres decorar la pantalla de controles.	Abrir asset

Mi recomendación práctica: usa Legacy Fantasy como fondo y estética principal. Si el paquete UI no combina al 100%, úsalo solo como referencia de paneles o reemplázalo por paneles sencillos creados por ti con sprites del propio bosque.

3.1. Qué descargar exactamente

- Del paquete Legacy Fantasy: backgrounds, trees, decorative props, stone/grass borders y cualquier sprite que te sirva para enmarcar la arena.
- Del Pixel UI Pack: botones para Jugar, Controles, Salir, Pausa, Revancha y Menú.
- De UI Audio: al menos 1 click para botones.
- De Impact Sounds: 1 sonido para rebote con paleta, 1 para rebote con borde y 1 para punto / gol.

3.2. Cómo debería verse tu juego al final



Mockup 1: pantalla de menú principal propuesta.



Mockup 2: gameplay final esperado del "Fantasy Pong".



Mockup 3: pantalla de victoria / cierre de partida.

4. Preparación del proyecto en Unity

15. Abrir Unity Hub.
16. Crear proyecto nuevo con plantilla 2D.
17. Nombre del proyecto: Ping-Pong_Game.
18. Crear el proyecto y dejar que Unity genere su estructura base.

4.1. Ajustes obligatorios para trabajar bien con Git

- Edit > Project Settings > Editor > Version Control Mode = Visible Meta Files.
- Edit > Project Settings > Editor > Asset Serialization = Force Text.
- No borres los archivos .meta de Unity.
- Para este proyecto usa Input clásico (Input.GetKey / Input.GetAxisRaw) porque es más rápido para un Pong académico.

4.2. Estructura de carpetas dentro de Assets

```
Assets/
├── _Project/
│   ├── Art/
│   │   ├── Backgrounds/
│   │   ├── Sprites/
│   │   ├── UI/
│   │   └── VFX/
│   ├── Audio/
│   │   ├── Music/
│   │   └── SFX/
│   ├── Materials/
│   ├── Prefabs/
│   ├── Scenes/
│   ├── Scripts/
│   │   ├── Core/
│   │   ├── Gameplay/
│   │   ├── UI/
│   │   └── Audio/
│   ├── Settings/
│   └── ThirdParty/
```

4.3. Escenas mínimas

```
Scenes/
├── Menu.unity
└── Game.unity
```

No mezcles todo en la raíz de Assets. Ordena el proyecto desde el día 1. Te ahorrará caos cuando tengas que exponer, buscar scripts o corregir algo rápido.

5. Git Flow simplificado para ti solo

```
main
└── develop
```

```
|— feature/project-setup
|— feature/assets-import
|— feature/level-design
|— feature/gameplay-core
|— feature/scoring-endgame
|— feature/ui-menu
|— feature/audio-polish
|— release/v1.0.0
```

Rama	Uso
main	Solo versiones estables y entregables.
develop	Integración del trabajo en progreso.
feature/project-setup	Estructura del proyecto, escenas vacías, .gitignore, ajustes base.
feature/assets-import	Importación y organización de assets.
feature/level-design	Construcción visual de la arena / cancha.
feature/gameplay-core	Movimiento de paletas, pelota y colisiones.
feature/scoring-endgame	Marcador, goles, reset de pelota y condición de victoria.
feature/ui-menu	Menú principal, pausa, controles y pantalla de victoria.
feature/audio-polish	Sonidos, partículas, correcciones visuales y pulido.
release/v1.0.0	Preparación del build final y cierre de entrega.

Regla de oro: nunca trabajes directamente en main. Siempre sales desde develop hacia una feature, terminas, haces commit, push, vuelves a develop y haces merge.

6. Crear el repositorio local y remoto desde cero

6.1. Crear el proyecto local y abrir terminal en la carpeta raíz

```
cd "C:\Ruta\al\tu\proyecto\Ping-Pong_Game"
```

6.2. Inicializar Git y crear el primer commit

```
git init -b main
# crea o pega un .gitignore de Unity
git add .
git commit -m "chore: initialize Unity project"
```

6.3. .gitignore recomendado para Unity

```
/[Ll]ibrary/
/[Tt]emp/
/[Oo]bj/
/[Bb]uild/
/[Bb]uilds/
/[Ll]ogs/
/[Uu]ser[Ss]ettings/
.vs/
*.csproj
*.sln
```

6.4. Crear el repositorio remoto en GitHub con tu usuario

```
gh auth login

gh repo create Developer-vic1/Ping-Pong_Game --public --source=. --remote=origin --push
```

Si prefieres privado, reemplaza `--public` por `--private`.

6.5. Crear la rama develop

```
git checkout -b develop
git push -u origin develop
```

Estado correcto al terminar este bloque: ya debes tener el proyecto Unity creado, Git inicializado, el remoto en GitHub creado como Developer-vic1/Ping-Pong_Game, main publicado y develop publicada.

7. Ruta de desarrollo completa: paso a paso + commit + push + merge

Fase	Qué haces	Commit de cierre sugerido
Fase 1 · feature/project-setup	Estructura del proyecto, escenas, carpetas, settings	chore: create folder structure and base scenes
Fase 2 · feature/assets-import	Descargar, importar y ordenar assets	chore: import visual, UI and audio assets
Fase 3 · feature/level-design	Construir la cancha / arena en la escena	feat: create fantasy pong arena

	Game	
Fase 4 · feature/gameplay-core	Movimiento, físicas y colisiones	feat: implement core pong gameplay
Fase 5 · feature/scoring-endgame	Puntos, marcador, reset, ganador	feat: add score system and endgame
Fase 6 · feature/ui-menu	Menú, controles, pausa y pantalla final	feat: add menu and gameplay UI
Fase 7 · feature/audio-polish	Sonidos, detalles visuales y fixes	feat: add audio and polish gameplay
Fase 8 · release/v1.0.0	Build, README, revisión final y tag	chore: prepare v1.0.0 release

7.1. Fase 1 · feature/project-setup

```
git checkout develop
git checkout -b feature/project-setup
```

- Crear todas las carpetas dentro de Assets/_Project.
- Crear las escenas Menu.unity y Game.unity.
- Guardar ambas escenas dentro de _Project/Scenes.
- Configurar Visible Meta Files y Force Text.
- Crear README.md inicial con nombre del proyecto y objetivo.

Commits recomendados dentro de esta fase:

```
git add .
git commit -m "chore: create project folder structure"
git add .
git commit -m "chore: create base menu and game scenes"
git add .
git commit -m "docs: add initial project README"
```

Cómo cerrar esta rama:

```
git push -u origin feature/project-setup

git checkout develop
git merge --no-ff feature/project-setup -m "merge: feature/project-setup into develop"
git push origin develop
git branch -d feature/project-setup
# opcional: git push origin --delete feature/project-setup
```

No avances hasta que... Las escenas deben existir, abrir sin errores y la estructura del proyecto debe quedar ordenada.

7.2. Fase 2 · feature/assets-import

```
git checkout develop
git checkout -b feature/assets-import
```

- Descargar el pack Legacy Fantasy - High Forest y descomprimirlo.
- Importar solo los sprites / tiles / backgrounds útiles, no todo si no lo necesitas.

- Importar UI sprites para botones o paneles.
- Importar sonidos de click, impacto y punto.
- Mover cada asset a su carpeta correcta en _Project.

Commits recomendados dentro de esta fase:

```
git add .
git commit -m "chore: import environment art assets"

git add .
git commit -m "chore: import ui assets"

git add .
git commit -m "chore: import gameplay and ui sounds"
```

Cómo cerrar esta rama:

```
git push -u origin feature/assets-import

git checkout develop
git merge --no-ff feature/assets-import -m "merge: feature/assets-import into develop"
git push origin develop
git branch -d feature/assets-import
# opcional: git push origin --delete feature/assets-import
```

No avances hasta que... Todos los assets deben verse en Project, sin referencias rotas, y ya ordenados por tipo.

7.3. Fase 3 · feature/level-design

```
git checkout develop
git checkout -b feature/level-design
```

- Entrar a la escena Game.unity.
- Crear Main Camera y ajustar la vista para un juego horizontal 2D.
- Añadir un fondo visual del bosque.
- Construir bordes superior e inferior con sprites de piedra/grama.
- Crear un centro visual con línea punteada o marca de mitad.
- Crear los postes laterales y preparar zonas de gol a izquierda y derecha.

```
Game Scene (Hierarchy)
|— Main Camera
|— Global Light 2D (si usas URP 2D)
|— Background
|— Arena
|   |— TopBorder
|   |— BottomBorder
|   |— CenterLine
|   |— LeftFrame
|   |— RightFrame
|— GoalLeft
|— GoalRight
```

Commits recomendados dentro de esta fase:

```
git add .
git commit -m "feat: add background and arena borders"
git add .
git commit -m "feat: add goals and arena center line"
```

Cómo cerrar esta rama:

```
git push -u origin feature/level-design

git checkout develop
git merge --no-ff feature/level-design -m "merge: feature/level-design into develop"
git push origin develop
git branch -d feature/level-design
# opcional: git push origin --delete feature/level-design
```

No avances hasta que... La cancha debe verse terminada visualmente, aunque todavía no funcione el gameplay.

7.4. Fase 4 · feature/gameplay-core

```
git checkout develop
git checkout -b feature/gameplay-core
```

- Crear los objetos PaddleLeft y PaddleRight.
- Crear el objeto Ball.
- Añadir SpriteRenderer y Collider2D a todos.
- A la pelota: Rigidbody2D Dynamic + CircleCollider2D.
- A las paletas: BoxCollider2D + Rigidbody2D Kinematic (o sin rigidbody si controlas por transform).
- Crear los scripts PaddleController.cs y BallController.cs.
- Programar controles W/S para Player 1 y Flecha Arriba/Abajo para Player 2.
- Programar velocidad inicial y rebotes de la pelota.

```
Scripts/Gameplay/
├─ PaddleController.cs
└─ BallController.cs
```

Commits recomendados dentro de esta fase:

```
git add .
git commit -m "feat: implement paddle movement"
git add .
git commit -m "feat: implement ball movement"
git add .
git commit -m "feat: add collisions with paddles and borders"
git add .
git commit -m "fix: clamp paddles inside arena"
```

Cómo cerrar esta rama:

```
git push -u origin feature/gameplay-core

git checkout develop
```

```
git merge --no-ff feature/gameplay-core -m "merge: feature/gameplay-core into develop"
git push origin develop
git branch -d feature/gameplay-core
# opcional: git push origin --delete feature/gameplay-core
```

No avances hasta que... Ya debes poder jugar una partida básica: las paletas se mueven y la pelota rebota correctamente.

7.5. Fase 5 • feature/scoring-endgame

```
git checkout develop
git checkout -b feature/scoring-endgame
```

- Crear triggers GoalLeft y GoalRight con BoxCollider2D (Is Trigger = true).
- Crear ScoreManager.cs.
- Crear GameManager.cs para controlar reinicio y final de partida.
- Cuando la pelota entra a GoalLeft: suma Player 2.
- Cuando la pelota entra a GoalRight: suma Player 1.
- Resetear la pelota al centro después de cada punto.
- Definir condición de victoria: por ejemplo, el primero en llegar a 5 gana.

```
Scripts/Core/
├── GameManager.cs
└── ScoreManager.cs

Scripts/Gameplay/
└── GoalTrigger.cs
```

Commits recomendados dentro de esta fase:

```
git add .
git commit -m "feat: implement score zones"
git add .
git commit -m "feat: add score manager and ui update"
git add .
git commit -m "feat: add endgame condition and ball reset"
```

Cómo cerrar esta rama:

```
git push -u origin feature/scoring-endgame

git checkout develop
git merge --no-ff feature/scoring-endgame -m "merge: feature/scoring-endgame into develop"
git push origin develop
git branch -d feature/scoring-endgame
# opcional: git push origin --delete feature/scoring-endgame
```

No avances hasta que... El juego debe contar puntos, reiniciar la pelota y detener o cerrar la partida cuando alguien gane.

7.6. Fase 6 • feature/ui-menu

```
git checkout develop
git checkout -b feature/ui-menu
```

- En Menu.unity: crear título del juego y botones Jugar / Controles / Salir.
- En Game.unity: crear HUD con Score Player 1 y Score Player 2.
- Crear panel de pausa con tecla ESC.
- Crear panel o escena de victoria al terminar la partida.
- Crear SceneLoader.cs o MainMenu.cs para cargar escenas y salir del juego.

```
Menu Scene
├─ Canvas
│   ├── Title
│   ├── ButtonPlay
│   ├── ButtonControls
│   └─ ButtonExit
Game Scene
├─ Canvas
│   ├── HUD
│   │   ├── ScoreP1
│   │   └─ ScoreP2
│   ├── PausePanel
│   └─ WinPanel
```

Commits recomendados dentro de esta fase:

```
git add .
git commit -m "feat: create main menu screen"
git add .
git commit -m "feat: add score hud and pause menu"
git add .
git commit -m "feat: add victory screen and scene transitions"
```

Cómo cerrar esta rama:

```
git push -u origin feature/ui-menu

git checkout develop
git merge --no-ff feature/ui-menu -m "merge: feature/ui-menu into develop"
git push origin develop
git branch -d feature/ui-menu
# opcional: git push origin --delete feature/ui-menu
```

No avances hasta que... Debes poder entrar desde Menu a Game, pausar, terminar la partida y volver al menú.

7.7. Fase 7 • feature/audio-polish

```
git checkout develop
git checkout -b feature/audio-polish
```

- Asignar sonido a rebote con paleta.

- Asignar sonido a rebote con borde.
- Asignar sonido a gol / punto.
- Asignar click a botones del menú.
- Pulir tamaños, posiciones y legibilidad de la UI.
- Añadir partículas o glow sencillo si quieres mejorar la presentación.
- Corregir cualquier bug encontrado al probar.

Commits recomendados dentro de esta fase:

```
git add .
git commit -m "feat: add gameplay sound effects"
git add .
git commit -m "feat: add ui click sounds"
git add .
git commit -m "fix: polish gameplay and menu visuals"
```

Cómo cerrar esta rama:

```
git push -u origin feature/audio-polish

git checkout develop
git merge --no-ff feature/audio-polish -m "merge: feature/audio-polish into develop"
git push origin develop
git branch -d feature/audio-polish
# opcional: git push origin --delete feature/audio-polish
```

No avances hasta que... El juego debe sentirse terminado: visualmente claro, con sonidos y sin fallos evidentes.

7.8. Fase 8 • release/v1.0.0

```
git checkout develop
git checkout -b release/v1.0.0
```

- Revisar que Menu y Game estén en Build Settings.
- Probar una partida completa de inicio a fin.
- Revisar la consola de Unity: 0 errores rojos.
- Completar o mejorar README.md.
- Crear el build final para Windows.
- Verificar que el .exe abra y funcione.

Build Settings / Build Profile

```
Scenes in Build:
0 - Menu.unity
1 - Game.unity
```

Commits recomendados dentro de la release:

```
git add .
git commit -m "docs: update README and project instructions"
git add .
git commit -m "chore: prepare final build configuration"
```

Cómo cerrar la release:

```
git push -u origin release/v1.0.0

git checkout main
git merge --no-ff release/v1.0.0 -m "release: v1.0.0"
git push origin main

git tag -a v1.0.0 -m "First stable release of Ping-Pong_Game"
git push origin v1.0.0

git checkout develop
git merge --no-ff release/v1.0.0 -m "merge: release/v1.0.0 into develop"
git push origin develop

git branch -d release/v1.0.0
```

No cierras la release si... el ejecutable no abre, la partida no finaliza, los botones no funcionan o hay errores rojos en consola.

8. Qué objetos debes tener sí o sí en la escena Game

Objeto	Componentes mínimos	Función
Main Camera	Camera	Ver el juego.
Background	SpriteRenderer	Fondo visual del bosque.
TopBorder	SpriteRenderer + BoxCollider2D	Rebote superior.
BottomBorder	SpriteRenderer + BoxCollider2D	Rebote inferior.
GoalLeft	BoxCollider2D (Trigger)	Cuando entra la pelota, anota Player 2.
GoalRight	BoxCollider2D (Trigger)	Cuando entra la pelota, anota Player 1.
PaddleLeft	SpriteRenderer + BoxCollider2D + PaddleController	Control del jugador 1.
PaddleRight	SpriteRenderer + BoxCollider2D + PaddleController	Control del jugador 2.
Ball	SpriteRenderer + CircleCollider2D + Rigidbody2D + BallController	Pelota del juego.
GameManager	GameManager + ScoreManager	Control general del flujo.
Canvas	Textos / botones / paneles	Marcador, pausa y victoria.

8.1. Scripts sugeridos

Script	Qué hace
PaddleController.cs	Mover una paleta hacia arriba/abajo y limitarla dentro de la arena.
BallController.cs	Mover la pelota, lanzar al inicio, reiniciar posición.
GoalTrigger.cs	Detectar si entró gol a izquierda o derecha.
ScoreManager.cs	Guardar y actualizar el puntaje de ambos jugadores.
GameManager.cs	Saber cuándo empieza, cuándo termina y mostrar victoria.
MainMenu.cs / SceneLoader.cs	Cargar Game desde Menu y volver al menú.
PauseMenu.cs	Pausar con ESC, reanudar y volver al menú.
AudioManager.cs	Reproducir SFX y clicks.

8.2. Controles recomendados

```
Jugador 1
W = subir
S = bajar

Jugador 2
Flecha Arriba = subir
Flecha Abajo = bajar

Sistema
ESC = pausa
```

9. README mínimo que debes dejar en GitHub

```
# Ping-Pong_Game

Proyecto académico realizado en Unity 2D.

## Características
- Menú principal
- Gameplay tipo Pong para 2 jugadores
- Marcador
- Sonidos
- Pausa
- Pantalla de victoria

## Controles
- Player 1: W / S
- Player 2: ↑ / ↓
- Pausa: ESC

## Tecnologías
- Unity 2D
- C#
- Git + GitHub

## Estado
v1.0.0 - estable
```

9.1. Convención de commits recomendada

Tipo	Ejemplo
chore	chore: initialize Unity project
feat	feat: add score system
fix	fix: clamp paddles inside arena
docs	docs: update README
release	release: v1.0.0

--	--

10. Checklist final antes de entregar

- ☐ El repositorio Ping-Pong_Game existe en tu GitHub.
- ☐ main tiene la versión estable.
- ☐ develop contiene también la versión final integrada.
- ☐ El proyecto abre en Unity sin errores graves.
- ☐ Menu.unity funciona.
- ☐ Game.unity funciona.
- ☐ La pelota se mueve.
- ☐ Las paletas responden a los controles.
- ☐ Las colisiones funcionan.
- ☐ El marcador suma correctamente.
- ☐ Hay sonido en gameplay y/o UI.
- ☐ Hay finalización de partida con ganador.
- ☐ El ejecutable .exe abre correctamente.
- ☐ README está subido y entendible.
- ☐ No subiste carpetas como Library o Temp.

11. Mini guion para tu exposición rápida

19. Empieza mostrando la pizarra o explicando rápidamente los requisitos: 2 escenas, colisiones, sonidos, contador, final y GitHub.
20. Muestra el repositorio y explica que trabajaste con Git Flow simplificado: main, develop y features.
21. Muestra el menú y entra al juego.
22. Juega una partida corta para enseñar movimiento, rebotes y marcador.
23. Haz que uno de los jugadores gane para enseñar la finalización.
24. Menciona brevemente los assets usados y que organizaste el proyecto por carpetas y ramas.

Frase final recomendada para cerrar tu presentación: “El proyecto se desarrolló en Unity 2D usando una metodología por etapas con Git Flow simplificado, incluyendo menú, gameplay completo, colisiones, sonidos, marcador, finalización de partida y repositorio en GitHub”.

12. Ruta resumida en una sola mirada

```
main
└─ develop
   ├── feature/project-setup
   ├── feature/assets-import
   ├── feature/level-design
   ├── feature/gameplay-core
   ├── feature/scoring-endgame
   ├── feature/ui-menu
   ├── feature/audio-polish
   └─ release/v1.0.0
```

Orden de trabajo:

- 1) Crear proyecto y repo
- 2) Crear develop
- 3) project-setup
- 4) assets-import
- 5) level-design
- 6) gameplay-core
- 7) scoring-endgame
- 8) ui-menu
- 9) audio-polish
- 10) release/v1.0.0
- 11) merge a main
- 12) tag v1.0.0

Si te atorras en algún punto: vuelve siempre al último estado estable en develop y revisa: “¿me faltó terminar la escena?, ¿me faltó asignar componentes?, ¿o me faltó conectar el script al objeto?” En proyectos Unity, la mitad de los errores vienen de no asignar componentes o referencias en el Inspector.